TECHNICAL REFERENCE GUIDE

Perpetual Weighted-Average Inventory Costing Cycle

Positive stock, negative-stock pending cost, FIFO revaluation, backdated replay, and current item/store balances

MiniErp | Final implementation reference

Version 1.0 | 29 July 2026

Implementation complete - migration creation pending explicit approval

Contents

Reading rule: Stock validation decides whether a quantity timeline is permitted. Inventory costing then assigns monetary cost to that permitted timeline. The two stages are related but must remain separate.

1. Purpose, scope, and invariants

This document defines the complete perpetual weighted-average inventory cost cycle implemented by MiniErp. It covers movement source costs, positive and negative quantity behavior, FIFO coverage of pending outbound cost, historical replay, current balances, transactions, contracts, and migration expectations.

18. Cost is calculated independently for each CompanyId + StoreId + ItemId partition.
19. Every active ItemMovement participates in chronological replay.
20. Average cost is calculated only when quantity is positive.
21. When QuantityAfter <= 0, AverageCostAfter and InventoryValueAfter are both zero.

22. Pending outbound quantity is never finalized at zero cost.
23. Purchase returns use the current weighted average, never the original purchase price.
24. Matching movement identity and CreatedOn are preserved during document updates.
25. Costing snapshots, allocations, and current balances are server-owned.

Non-goals: Inventory costing does not introduce document status, posting, cancellation, reversal, general-ledger entries, vouchers, or payment allocations.

2. Partition key and persisted model

```
InventoryCostingKey = CompanyId + StoreId + ItemId
```

All quantity, average-cost, inventory-value, pending-cost, allocation, locking, and replay decisions are isolated by this tenant-safe key.

2.1 Persisted entities

Entity	Purpose	Important persisted fields
ItemMovement	Canonical chronological quantity and cost event.	UnitCost; TotalCost; QuantityAfter; AverageCostAfter; InventoryValueAfter; PendingCostQuantity; CostStatus.
InventoryCostAllocation	Derived audit row linking future inbound cost to an earlier pending outbound.	CompanyId; StoreId; ItemId; OutboundMovementId; InboundMovementId; Quantity; UnitCost; TotalCost; CreatedOn.
ItemStoreBalance	Current server-maintained projection for one item/store.	Quantity; AverageCost; InventoryValue; RowVersion.
InvoiceLine	Invoice input and optional linked sales-return source.	Price; SourceInvoiceLineId; ReturnUnitCost.
StockAdjustmentLine	Manual adjustment input.	UnitCost is required only for an Increase.

2.2 Keys, indexes, and constraints

Object	EF Core / database design	Reason
ItemMovement	Primary key Id; alternate key (CompanyId, Id).	Supports tenant-safe composite allocation foreign keys.
ItemMovement timeline	Filtered index on CompanyId, StoreId, ItemId, MovementDate, CreatedOn, Id.	Deterministic active replay.

Object	EF Core / database design	Reason
Pending movements	Filtered index includes CostStatus and chronological columns.	Locates Pending and PartiallyCosted movements.
InventoryCostAllocation	Unique CompanyId, OutboundMovementId, InboundMovementId.	One rebuilt allocation per inbound/outbound pair.
ItemStoreBalance	Composite primary key CompanyId, StoreId, ItemId; RowVersion.	One concurrent current balance per costing partition.
InvoiceLine source	Tenant-safe optional self-reference by CompanyId and SourceInvoiceLineId.	Prevents cross-company return-cost linkage.

Database checks enforce nonnegative monetary cost, pending quantity not exceeding QuantityOut, exactly one movement direction, and zero average/value whenever the running quantity is nonpositive.

3. Precision, rounding, and server ownership

Value family	SQL precision	Rounding / ownership
Quantity	decimal(18,6)	AwayFromZero; input quantity is validated.
Unit and average cost	decimal(24,8)	AwayFromZero; source input only where explicitly allowed.
Total cost and inventory value	decimal(28,8)	AwayFromZero; always server-calculated.

3.1 Client-entered cost fields

26. Purchase invoice line Price.
27. Opening-balance line Price, presented to the user as Unit Cost.
28. Stock-adjustment Increase UnitCost.
29. Unlinked Sales Return ReturnUnitCost only when no positive current average exists.

3.2 Server-calculated fields

30. ItemMovement CostStatus, PendingCostQuantity, UnitCost for outbound movements, TotalCost, QuantityAfter, AverageCostAfter, and InventoryValueAfter.
31. Every InventoryCostAllocation field.
32. ItemStoreBalance Quantity, AverageCost, InventoryValue, and RowVersion.
33. All cost snapshots returned by invoice, adjustment, and opening-balance APIs.

Important: A Stock Adjustment Decrease must omit UnitCost. The server uses the weighted average immediately before that outbound movement.

4. Company stock-check policy

Each company selects one StockBalanceCheckMode. This controls whether the quantity timeline may become negative; it does not disable costing.

Mode	Quantity validation	Costing behavior
None	No negative-stock rejection.	Replay and pending-cost logic still run.
DateCheck	Reject any negative point in the chronological timeline.	Replay runs only after validation succeeds.
FinalCheck	Reject only a negative resulting final balance.	Earlier negative points may be costed through pending FIFO.
Both	Apply DateCheck and FinalCheck.	Replay runs only after both checks succeed.

Outbound create, update, and delete run the configured check. Inbound update and delete also run it because they may remove quantity that supports later outbound movements. A new inbound create skips stock validation because it only adds quantity, but it still triggers costing replay.

5. Complete movement-processing cycle

34. Start one Serializable database transaction for the document operation.
35. Validate request shape, tenant ownership, active product store, active item and unit, and document header RowVersion where applicable.
36. Derive every old and new CompanyId + StoreId + ItemId key affected by the change.
37. Lock balance keys in deterministic CompanyId, StoreId, ItemId order. SQL Server uses UPDLOCK and HOLDLOCK, including a range lock when no balance row exists.
38. Run the company stock-check policy against the exact proposed quantity timeline.
39. Reconcile document movements while preserving matching ItemMovement.Id and CreatedOn.
40. Load the full active movement timeline for each affected key ordered by MovementDate, CreatedOn, Id.
41. Physically remove the affected derived InventoryCostAllocation rows.
42. Replay every inbound and outbound movement, rebuild pending FIFO allocations, and refresh all movement snapshots.

43. Apply the last replay state to ItemStoreBalance.
44. Save the document, movements, snapshots, allocations, and balances and commit. Any failure rolls everything back.

Separation of concerns: Stock validation evaluates quantity permission. Costing replay evaluates monetary value. Validation must finish before final costing is saved.

6. Inbound and outbound cost-source rules

6.1 Inbound movements

Movement	Unit-cost source	Special rule
Purchase	Purchase invoice line Price.	Recalculates weighted average.
Opening balance	Opening-balance line Price.	Creates an OpeningBalance ItemMovement.
Adjustment Increase	Required StockAdjustmentLine.UnitCost.	Client cost must be nonnegative.
Linked Sales Return	Final UnitCost of the original Sales movement.	Source sale must precede the return and be fully costed.
Unlinked Sales Return	Current positive average; otherwise ReturnUnitCost.	Fallback is required only without a positive average.
Transfer In	Stored source movement cost.	Rejected when no source cost is available.

6.2 Outbound movements

45. Sales use the current weighted average immediately before the movement.
46. Stock Adjustment Decrease uses the current weighted average immediately before the movement.
47. Purchase Return uses the current weighted average immediately before the movement and never the original purchase price.
48. An outbound movement does not change a positive average unless remaining quantity becomes zero or negative.

6.3 Positive-stock formulas

```
InboundTotalCost      = QuantityIn * SourceUnitCost
QuantityAfter         = PreviousQuantity + QuantityIn
InventoryValueAfter   = PreviousInventoryValue + InboundTotalCost
AverageCostAfter      = InventoryValueAfter / QuantityAfter
Condition: calculate average only when QuantityAfter > 0
```

```
OutboundUnitCost     = PreviousAverageCost
OutboundTotalCost    = QuantityOut * OutboundUnitCost
QuantityAfter        = PreviousQuantity - QuantityOut
InventoryValueAfter  = PreviousInventoryValue - OutboundTotalCost
```

7. Negative-stock pending-cost lifecycle

When policy permits an outbound quantity greater than available positive stock, the movement is logically divided into covered and pending portions.

```
CoveredQuantity      = min(QuantityOut, max(PreviousQuantity, 0))
PendingCostQuantity  = QuantityOut - CoveredQuantity
CoveredCost          = CoveredQuantity * PreviousAverageCost
```

7.1 Cost status

Status	Meaning	UnitCost rule
Final	Completely costed without future dependency.	Populated.
PartiallyCosted	Some outbound quantity is costed and some remains pending.	Null until fully covered.
Pending	The full outbound quantity awaits future inbound cost.	Null.
Revalued	A previously pending movement is now fully covered.	TotalCost / QuantityOut.

7.2 FIFO coverage by later inbound

49. Pending outbound movements are queued by MovementDate, CreatedOn, Id.
50. A future inbound first covers the earliest pending outbound quantity.
51. Each cover writes one InventoryCostAllocation with quantity, source unit cost, and total cost.
52. Only the remaining inbound quantity enters positive inventory and participates in the new average.
53. When the pending quantity reaches zero, the outbound becomes Revalued and receives its final effective UnitCost.

Hard invariant: When QuantityAfter <= 0, AverageCostAfter = 0 and InventoryValueAfter = 0. Never divide inventory value by a zero or negative quantity.

8. Worked costing examples

8.1 Positive weighted-average cycle

Start with 10 units at cost 5.00. Purchase 10 units at 7.00, sell 8 units, increase by 3 units at 9.00, then return 5 units to the supplier.

Movement	Calculation	Snapshot after movement
Opening 10 @ 5	Value = 50	Qty 10; Avg 5; Value 50
Purchase 10 @ 7	New value = 50 + 70	Qty 20; Avg 6; Value 120
Sale 8	COGS = 8 * 6 = 48	Qty 12; Avg 6; Value 72
Adjustment Increase 3 @ 9	New value = 72 + 27	Qty 15; Avg 6.6; Value 99
Purchase Return 5	Cost = 5 * 6.6 = 33	Qty 10; Avg 6.6; Value 66

8.2 Partial negative stock and revaluation

Previous quantity is 3 at average cost 10. An outbound movement issues 5 units. A later purchase brings 10 units at cost 12.

```
Outbound: covered 3 * 10 = 30; pending 2; QuantityAfter = -2
Snapshot: TotalCost = 30; UnitCost = null; CostStatus = PartiallyCosted
Inbound: allocate 2 * 12 = 24; remaining inbound = 8
Revalued outbound: TotalCost = 54; UnitCost = 54 / 5 = 10.8
Final balance: Quantity = 8; AverageCost = 12; InventoryValue = 96
```

8.3 Multiple inbound covers

An outbound of 6 units occurs from zero stock. Later inbound movements arrive as 2 @ 10, 3 @ 12, and 4 @ 15.

```
After outbound: Pending = 6; TotalCost = 0
Inbound 2 @ 10: allocate 20; Pending = 4
Inbound 3 @ 12: allocate 36; Pending = 1; accumulated cost = 56
Inbound 4 @ 15: allocate 15; outbound TotalCost = 71
Final outbound UnitCost = 71 / 6 = 11.83333333
Remaining positive stock = 3 @ 15; InventoryValue = 45
```

8.4 Accounting reconciliation

```
Opening inventory value + total inbound value
= final inventory value + total fully costed outbound cost
```

The equation closes when all outbound movements are fully costed. If Pending or PartiallyCosted movements remain, the unresolved quantity is reported explicitly and the final cost equation is not yet closed.

9. Backdated replay and stable movement identity

9.1 Replay triggers

Full replay is required after any of: create; edit; delete; restore; movement-date change; quantity change; cost change; item change; store change; company change; direction change; movement-type change; source-linkage change; soft-delete change.

Every old and new affected partition is replayed. Replay includes all subsequent inbound and outbound movements, not only inbound movements. When the safe dependency boundary is uncertain, MiniErp replays the full active timeline for the item/store.

9.2 Stable movement identity

54. Update matching movements in place.
55. Preserve matching ItemMovement.Id and CreatedOn.
56. Soft-delete movements for removed lines.
57. Create a new movement only for a newly added line.
58. Never delete and recreate all document movements during a normal update.

9.3 Allocation rebuild

InventoryCostAllocation is server-derived and rebuildable. Replay physically deletes the affected allocation rows, then rebuilds them deterministically. The unique movement-pair constraint prevents ambiguous duplicate allocations.

10. Return linkage rules

10.1 Linked Sales Return

59. SourceInvoiceLineId must identify an active Sales line in the same company, store, and item.
60. The source Sales movement must precede the return in deterministic movement order.
61. The source movement must be Final or Revalued and have a non-null UnitCost.
62. Pending and PartiallyCosted source sales are rejected.
63. An active linked sales return blocks soft deletion of its source Sales invoice.

10.2 Unlinked Sales Return

Use the current positive average when available. Require ReturnUnitCost only when no positive average exists. ReturnUnitCost is a fallback source input, not a server snapshot field.

10.3 Purchase Return

> **Approved policy:** A purchase return is a normal outbound movement. It uses the current weighted average immediately before the movement and never uses the original purchase price.

11. Atomicity, locking, and concurrency

11.1 Transaction boundary

Invoices, stock adjustments, opening balances, and inventory-count reconciliation execute their movement-producing workflows inside one Serializable SQL Server transaction. A failure in validation, document concurrency, costing, allocation rebuilding, or balance persistence rolls the complete operation back.

11.2 Deterministic balance locking

Affected keys are ordered by CompanyId, StoreId, ItemId. Existing ItemStoreBalance rows are locked with UPDLOCK and HOLDLOCK. An exact key lookup under Serializable isolation range-locks a missing key so two writers cannot independently create the same balance.

11.3 Concurrency tokens

64. Document aggregates retain their existing header RowVersion contract.
65. The client must send the originally returned header token; EF Core uses that exact token as OriginalValue.
66. Line-only aggregate changes touch the header so its RowVersion advances.
67. ItemStoreBalance has its own SQL Server RowVersion for current-balance write conflicts.
68. DbUpdateConcurrencyException rolls back the transaction and returns the feature's Arabic reload-and-retry conflict.

12. API, Swagger, seed, and frontend contracts

12.1 API response fields

Invoice, stock-adjustment, and opening-balance line responses expose movement CostStatus, PendingCostQuantity where applicable, UnitCost, TotalCost, QuantityAfter, AverageCostAfter, and

InventoryValueAfter. Invoice item-balance responses expose current Balance, AverageCost, and InventoryValue.

12.2 Request rules

69. Stock Adjustment Increase: UnitCost is required for every line.
70. Stock Adjustment Decrease: UnitCost must be omitted.
71. Inventory Count reconciliation: IncreaseCosts must contain exactly one UnitCost for each positive difference and no extras.
72. Sales Return: optional SourceInvoiceLineId; otherwise ReturnUnitCost only when no positive average exists.
73. Server-calculated costing fields are never accepted from the client.

12.3 Frontend behavior

74. Company screen keeps company information and stock options in separate tabs; the stock-check dropdown has exactly four modes.
75. Every paginated Stock Adjustment item includes the complete deterministic line collection.
76. Adjustment, invoice, and opening-balance screens show unit cost, average cost, and inventory value.
77. Inventory Count requests a unit cost for each positive adjustment before reconciliation.
78. Sales Return supports source-line linkage and fallback ReturnUnitCost.

12.4 Seed behavior

Development seed ensures opening-balance movements exist, updates matching seeded movements in place, then runs full costing replay for each seeded company to populate movement snapshots, allocations, and current balances idempotently.

13. Migration and historical-data backfill plan

> **Current status:** The entity and EF Core configuration are implemented. No perpetual costing migration has been created or applied. Migration generation requires a separate explicit approval.

79. Add nullable UnitCost and required snapshot/status fields to ItemMovements with safe temporary defaults for existing rows.
80. Create InventoryCostAllocations with tenant-safe composite foreign keys and the unique movement-pair constraint.

81. Create ItemStoreBalances with the composite primary key, monetary checks, and SQL Server RowVersion.
82. Add SourceInvoiceLineId and ReturnUnitCost to InvoiceLines and UnitCost to StockAdjustmentLines.
83. Backfill one stable OpeningBalance ItemMovement for every active opening-balance line that has no matching movement.
84. Resolve source cost for every historical inbound movement from its document line.
85. Replay every active CompanyId + StoreId + ItemId timeline in deterministic order.
86. Populate all movement snapshots, rebuild pending allocations, and create current ItemStoreBalance rows.
87. Validate nonnegative-cost and nonpositive-state constraints before enabling them against historical data.
88. Review Up, Down, generated snapshot, indexes, foreign keys, and existing-data SQL before applying the migration.

14. Verification matrix

Area	Required scenarios
Positive costing	Weighted inbound average; outbound current average; quantity to zero; average/value reset.
Negative costing	Fully pending; partially costed; one/multiple inbound covers; one inbound covers multiple outbounds; exact zero and positive recovery.
Replay	Backdated create; quantity/cost/date edit; item/store move; delete; restore; full later inbound/outbound recalculation.
Identity	Matching updates preserve movement Id and CreatedOn; removed lines soft-delete movements.
Returns	Purchase return current average; linked sale final/revalued; pending-source rejection; unlinked fallback.
Allocations	Deterministic rebuild; unique pair; FIFO chronology; correct effective outbound UnitCost.
Balance	Current balance equals final movement snapshot; quantity/value and accounting equation reconcile.
Cross-cutting	Tenant isolation; active store/item validation; transaction rollback; stale RowVersion; Arabic errors; Swagger; frontend build.

Verification recorded when this guide was prepared: 499 backend tests passed, .NET formatting verification passed, and the frontend production build passed.

15. Operational checklist

89. Check: Confirm the document transaction uses Serializable isolation.
90. Check: Derive both old and new item/store keys before changing movements.
91. Check: Acquire balance locks in deterministic order before stock validation and movement writes.
92. Check: Run the company's selected stock-check mode.
93. Check: Preserve matching movement identity and CreatedOn.
94. Check: Replay MovementDate, CreatedOn, Id across all later active movements.
95. Check: Rebuild affected allocation rows deterministically.
96. Check: Confirm nonpositive quantity has zero average and inventory value.
97. Check: Confirm pending outbound UnitCost remains null.
98. Check: Confirm successful responses return refreshed snapshots and RowVersion.
99. Check: Confirm rollback leaves no partial document, movement, allocation, or balance state.
100. Check: Do not create or apply a migration until the generated design is approved.

Appendix A. Arabic business messages

The following messages are part of the implemented business contract.

`Inventory.SalesReturnSourceCostPending`
لا يمكن احتساب تكلفة مرتجع البيع لأن حركة البيع الأصلية لم تكتمل تكلفتها بعد.

`Inventory.ReturnUnitCostRequired`
يجب إدخال تكلفة وحدة مرتجع البيع عند عدم توفر متوسط تكلفة موجب.

`StockAdjustments.UnitCostRequired`
يجب إدخال تكلفة الوحدة لكل سطر عند زيادة المخزون.

`StockAdjustments.UnitCostNotAllowed`
لا يجوز إدخال تكلفة الوحدة في تسوية الخصم؛ يستخدم الخادم متوسط التكلفة الحالي.

`Inventory.MovementCostSourceMissing`
تعذر العثور على مصدر تكلفة حركة المخزون.

Appendix B. Implementation reference

101. MiniErp.Domain/Entities/Inventory/ItemMovement.cs
102. MiniErp.Domain/Entities/Inventory/InventoryCostAllocation.cs
103. MiniErp.Domain/Entities/Inventory/ItemStoreBalance.cs
104. MiniErp.Infrastructure/Services/Inventory/InventoryCostingService.cs
105. MiniErp.Infrastructure/Persistence/Configurations/ItemMovementConfiguration.cs
106. MiniErp.Infrastructure/Persistence/Configurations/InventoryCostAllocationConfiguration.cs
107. MiniErp.Infrastructure/Persistence/Configurations/ItemStoreBalanceConfiguration.cs
108. MiniErp.Infrastructure/Services/Invoices/InvoiceService*.cs
109. MiniErp.Infrastructure/Services/StockAdjustments/StockAdjustmentService.cs
110. MiniErp.Infrastructure/Services/StockOpeningBalances/StockOpeningBalanceService.cs
111. MiniErp.Infrastructure/Services/InventoryCounts/InventoryCountService.cs
112. FEATURE_DEVELOPMENT_GUIDE.md - Perpetual weighted-average inventory costing